

Adora — MLP Equation of State

As-built notes · Ivan Milic · 2026-09-19 · session summary

Built [eos_mlp/](#): a tiny differentiable MLP surrogate for the **LTE equation of state**, so the electron density `ne` and total hydrogen density `nHtot` are no longer passed as free, degenerate RT inputs but recovered from the real thermodynamic state. Trained once on a Wittmann grid; inference is pure JAX and portable (no `lightweaver` at use time).

1. Concept

The MLP learns a single scalar, the electron pressure `Pe`, versus two thermodynamic variables. The primary head is $(T, P_{\text{gas}}) \rightarrow P_e$ (used for *both* synthesis and inversion); a second $(T, \rho) \rightarrow P_e$ head is kept as an optional convenience. Everything the RT needs then follows algebraically:

$$n_e = \frac{P_e}{k_B T}, \quad n_{\text{H,tot}} = \frac{P_{\text{gas}} - P_e}{k_B T \Sigma_A}, \quad \Sigma_A = \sum_i n_i / n_{\text{H}} = 1.095.$$

This replaces the two loosely-constrained free parameters $(n_e, n_{\text{H,tot}})$ with one physical variable, P_{gas} , per depth.

2. Ground truth & grid

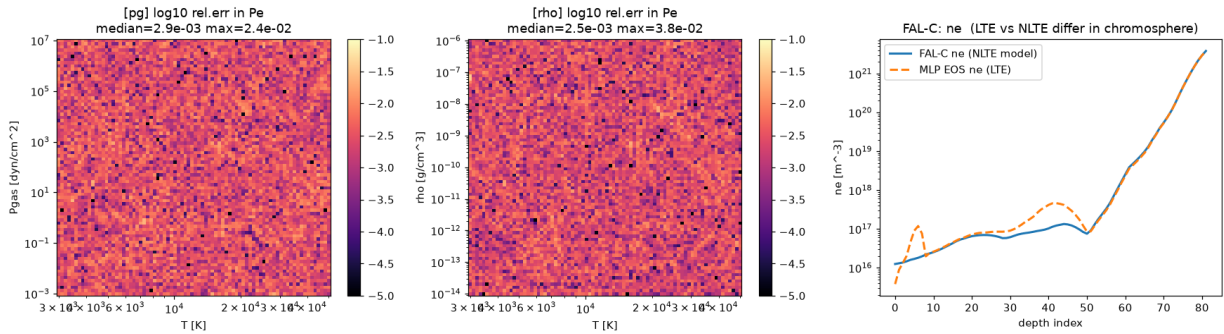
Ground truth is `lightweaver.wittmann.Wittmann` — a full multi-element LTE Saha–Boltzmann EOS (Kurucz partition functions, metal electron donors, molecules; CGS, scalar). Tabulated on a 96×96 grid over $T \in [3 \times 10^3, 5 \times 10^4]$ K and $P_{\text{gas}} \in [10^{-3}, 10^7]$ dyn/cm² (photosphere + chromosphere regime), 0 bad corners, ~ 25 s. `lightweaver` is a **data-generation-time dependency only**; the trained model is pure JAX.

3. Model & training

Deliberately small: random Fourier features ($N=32$) $\rightarrow$ MLP 64×64 , `tanh`, fp64. Fits $\log_{10} P_e$ in normalized CGS log space (SI is converted at the API boundary). Trained with `optax` Adam, 20k steps/head, ~ 4 min on CPU.

4. Accuracy — validated

Relative error in P_e (equivalently n_e , since $n_e \propto P_e$): **median 0.44%** (P_{gas} head) / 0.35% (ρ head), max ~ 3 –4% at the sharp hydrogen-ionization ridge; the error map is spatially uniform (fit noise, no systematic miss). Against FAL-C the model reproduces n_e to **<0.1%** in the photosphere where LTE holds, and diverges only in the upper chromosphere where FAL-C's n_e is NLTE hydrogen-ionization-enhanced — as an LTE EOS must. Gradients are finite (`jax.grad` verified); the API is `jit/vmap/grad`-safe.



Left/middle: $\log_{10}$ relative error in P_e vs Wittmann across the grid (`pg`, `rho` heads). Right: FAL-C n_e — MLP (LTE) tracks the model in the photosphere and deep layers, differs in the NLTE chromosphere as expected.

5. Files (all in [eos_mlp/](#))

[eos_config.py](#) (ranges/units), [generate_grid.py](#) (Wittmann $\rightarrow$ grids; needs `lightweaver`), [eos_model.py](#) (pure-JAX MLP), [train_eos.py](#) (one-time training), [eos.py](#) (portable inference, SI in/out), [validate_eos.py](#), [README.md](#). Artefacts: [eos_grid-{pg,rho}.npz](#), [eos-{pg,rho}.npz](#), [eos_validation.png](#).

6. Use & next

`ne, nHtot = eos.ne_nhtot_from_pg(eos.load_eos("pg"), T, Pgas)`, then feed `lte.polarised_rt(...)` — the RT needs **no change** (it already takes `ne, nhtot` in SI). **Next:** wire the `pg` head into the inversion (one `Pgas` node per depth in place of free `ne/nhtot`) and into [synth3d.py](#). Files are currently uncommitted.